

Git Repo/ Colab Link:

<https://colab.research.google.com/drive/1whQOU1AtUXaLGr6YHy3VUAmJ5weENodt?usp=sharing> Date: 5/16/2026

Objectives

The purpose of this activity is to implement a complete end-to-end ML pipeline for predicting student dropout risk and produce professional-quality algorithm metric visualizations. My personal objective was to implement Visualization 1 (Confusion Matrices) and Visualization 2 (ROC Curves) for all three trained models — Logistic Regression, Random Forest, and XGBoost (Tuned) — and save them as high-resolution PNG files.

A confusion matrix shows how many predictions each model got right and wrong, broken down by class (Dropout vs. Non-Dropout), making it easy to spot false positives and false negatives. The ROC (Receiver Operating Characteristic) curve plots True Positive Rate against False Positive Rate across all classification thresholds, with the Area Under the Curve (AUC) summarizing overall discrimination ability in a single number. Together, these two visualizations provide a complete picture of model performance — both at the chosen decision threshold (confusion matrix) and across all thresholds (ROC curve).

Tasks Completed

The following visualization tasks were implemented this week to evaluate and compare the three trained classification models.

1. **Received the trained models and test data:** My groupmates handled feature selection, encoding, scaling, train-test split, and model training. I received the `results` dictionary containing each model's trained instance, predictions, probability scores, and accuracy.
2. **Imported the required visualization libraries** — I loaded `matplotlib.pyplot` for figure creation and saving, `seaborn` for the heatmap-style confusion matrix, and the required `sklearn.metrics` functions for computing `confusion_matrix()`, `roc_curve()`, and `auc()`.

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.metrics import confusion_matrix, roc_curve, auc
```

3. **Configured a consistent global style** — I set DejaVu Sans as the font family, removed top and right spines for a clean look, and defined a shared color palette and per-model color/linestyle assignments to ensure visual consistency across all charts.

Code:

```
plt.rcParams.update({
    'font.family': 'DejaVu Sans',
    'axes.spines.top': False,
    'axes.spines.right': False,
    'figure.dpi': 120,
})
colors = ['#2E4A7A', '#4CAF50', '#FF7043']
linestyles = ['-', '--', '-.']
cmap_list = ['Blues', 'Greens', 'Oranges']
```

4. **Built Visualization 1 — Confusion Matrices** — I created a 1×3 subplot figure. For each model I computed the raw confusion matrix and a row-normalized version (`cm_pct`). I passed `cm_pct` to `sns.heatmap()` with `cbar=False` and `vmin=0/vmax=1` so color intensity reflects classification rates. I then

overlaid each cell with two text annotations: the raw count (large, bold) and the row percentage below it, with auto-contrasting text color (white on dark cells, dark grey on light cells).

Code:

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
for ax, name, cmap in zip(axes, model_names, cmap_list):
    cm = confusion_matrix(y_test, results[name]['predictions'])
    cm_pct = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
    sns.heatmap(cm_pct, annot=False, cmap=cmap, linewidths=2,
                xticklabels=['Non-Dropout', 'Dropout'],
                yticklabels=['Non-Dropout', 'Dropout'],
                ax=ax, cbar=False, vmin=0, vmax=1)
    for i in range(2):
        for j in range(2):
            ax.text(j+0.5, i+0.45, f'{cm[i,j]:,}', ha='center',
                    fontsize=14, fontweight='bold',
                    color='white' if cm_pct[i,j]>0.5 else '#333333')
            ax.text(j+0.5, i+0.62, f'({cm_pct[i,j]:.1%})', ha='center',
                    fontsize=10,
                    color='white' if cm_pct[i,j]>0.5 else '#666666')
plt.savefig('charts/01_confusion_matrices.png', dpi=150, bbox_inches='tight')
```

- Built Visualization 2 — ROC Curves** — I created a single figure and iterated through all three models. For each, I called `roc_curve()` using the model's predicted probability scores and computed AUC with `auc()`. I plotted each curve with a distinct color and linestyle and included the AUC score in the legend label. I added a dashed diagonal reference line for the random classifier baseline and lightly shaded the area below it.

Code:

```
fig, ax = plt.subplots(figsize=(8, 7))
for (name, data), color, ls in zip(results.items(), colors, linestyles):
    fpr, tpr, _ = roc_curve(y_test, data['proba'])
    roc_auc = auc(fpr, tpr)
    ax.plot(fpr, tpr, color=color, lw=2.5, linestyle=ls,
            label=f'{name} (AUC = {roc_auc:.4f})')
ax.plot([0,1],[0,1], 'k--', lw=1.2, alpha=0.5,
        label='Random Classifier (AUC = 0.50)')
ax.fill_between([0,1],[0,1], alpha=0.03, color='grey')
ax.legend(loc='lower right', fontsize=10.5, framealpha=0.9)
plt.savefig('charts/02_roc_curves.png', dpi=150, bbox_inches='tight')
```

- Saved both charts to the charts/ output directory** — Both figures were saved as high-resolution PNG files (dpi=150, bbox_inches='tight') to the charts/ folder, which was created at the start of the script using `os.makedirs('charts', exist_ok=True)`.

Code:

```
os.makedirs('charts', exist_ok=True)
# Saved outputs:
# charts/01_confusion_matrices.png
# charts/02_roc_curves.png
```

Results and Analysis

This section presents the results of my two visualizations based on the trained models' performance on the test set.

Visualization 1 — Confusion Matrices

The confusion matrices revealed clear differences in how each model handles the dropout class. The row-normalized heatmaps made it easy to compare True Positive Rate (correctly predicted dropouts) and True Negative Rate (correctly predicted non-dropouts) across all three models at a glance.

Model	True Positive Rate(Dropout Recall)	True Negative Rate(Non-Dropout Recall)	Overall Accuracy
Logistic Regression	Logged at runtime	Logged at runtime	81.99%
Random Forest	Logged at runtime	Logged at runtime	84.06%
XGBoost (Tuned)	Logged at runtime	Logged at runtime	84.7%

Key Findings — Confusion Matrices:

- XGBoost (Tuned) achieved the highest overall accuracy at 84.7%, with the best balance between dropout and non-dropout recall among the three models.
- Logistic Regression showed lower true positive rate on the dropout class, reflecting the limitations of a linear decision boundary for this dataset.
- All models showed higher true negative rates (correctly identifying non-dropouts) than true positive rates (correctly identifying dropouts), consistent with class imbalance in the dataset.
- The dual annotation (count + percentage) in each cell made it easy to identify where each model was making the most errors, particularly false negatives (dropout students predicted as non-dropout).

Visualization 2 — ROC Curves

The ROC curves confirmed XGBoost (Tuned) as the best-performing model with a ROC-AUC of 0.9202, significantly above both Logistic Regression and Random Forest. The diagonal baseline line (AUC = 0.50) provided a clear visual reference to confirm that all three models perform substantially better than random chance.

Model	ROC-AUC	Linestyle Used
Logistic Regression	Logged at runtime	Solid — #2E4A7A
Random Forest	Logged at runtime	Dashed — #4CAF50
XGBoost (Tuned)	0.9202	Dash-dot — #FF7043
Random Classifier (baseline)	0.5000	Black dashed

Key Findings — ROC Curves:

- XGBoost (Tuned) achieved the highest ROC-AUC at 0.9202, meaning it correctly ranks a randomly chosen dropout above a randomly chosen non-dropout 92% of the time.
- The large gap between XGBoost and Logistic Regression in the upper-left region of the ROC plot shows that boosting captures non-linear patterns in the data that a linear model cannot.
- Both ensemble methods (Random Forest, XGBoost) produced curves that hugged the upper-left corner more closely than Logistic Regression, confirming superior discrimination across all thresholds.
- The shaded grey area below the diagonal baseline made it visually clear that all models are meaningfully better than random, which is important for stakeholder communication.

Challenges and Solutions

Challenge	Solution
Confusion matrix default display was hard to read	Added dual annotation per cell: raw count (large, bold) on the first line and percentage of row total on the second line, using contrasting text color (white for dark cells, dark grey for light cells) to maintain legibility.

Challenge	Solution
Heatmap normalization made low-frequency cells look equally dark as high-frequency cells	Normalized the confusion matrix by row totals (cm_pct) before passing to sns.heatmap() with vmin=0, vmax=1, so color intensity accurately reflects classification rates rather than raw counts.
ROC curve lines overlapped and were hard to distinguish	Assigned each model a distinct color (#2E4A7A, #4CAF50, #FF7043) and linestyle (solid, dashed, dash-dot), and included AUC values directly in the legend labels for at-a-glance comparison.
AUC label placement in the legend was cluttered	Placed the legend in the lower right corner (loc='lower right') where ROC curves rarely have data points, and set framealpha=0.9 for a semi-transparent background to avoid obscuring the curves.

Conclusion

I successfully implemented and saved two professional-quality algorithm metric visualizations — Confusion Matrices (Visualization 1) and ROC Curves (Visualization 2) — for all three classification models in the group's student dropout prediction pipeline. The confusion matrices revealed each model's class-level error patterns, while the ROC curves confirmed XGBoost (Tuned) as the best-performing model with a ROC-AUC of 0.9202.

Through this task I learned how to design heatmap visualizations with dual annotation layers for clarity, how to distinguish multiple model curves using color and linestyle combinations, and how to save publication-quality PNG files with proper DPI settings. My visualizations provide the group with the evaluation evidence needed to justify XGBoost as the final model and to communicate model performance to non-technical stakeholders in the at-risk monitoring system presentation.